

Task 7 Attempt Summary

In this we were supposed to research and develop an extension to our project. In my case I realized that the project specification stated that this was meant to be a stock investment tool, so I decided on giving it the capability to analyze **Buy and sell signals**, with the help of decision trees and coding it as a **Classification model** because what we are doing is **binary classification**

0's represents → Sell signals

1's represents → Buy signals

The set of libraries used in this extension was,

```
import numpy as np
import datetime
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn import tree
```

The scikit learn “**tree**” library was used to represent the already trained and tested decision tree in a visual format.

The scikit learn “**classification report**” library was used to assess the accuracy of the decision tree model more on how the assessment works is in the later part of this report.

```
start = datetime.datetime(2010,1,1)
end = datetime.datetime.now()
```

Then I initialized 2 variables with the date that the training data should be downloaded from, and the current date.

```
stock_ticker = 'META'
df = yf.download(stock_ticker, start=start, end=end)
df.head(25)
```

Almost all the stock data is received through the company's ticker symbol, in my case its **META**, the company which owns Facebook, Instagram, WhatsApp etc....

I download the stock data into the data frame variable (“**df**”) using the **yahoo finance API**,

[*****100%*****] 1 of 1 completed

	Open	High	Low	Close	Adj Close	Volume
Date						
2012-05-18	42.049999	45.000000	38.000000	38.230000	38.115242	573576400
2012-05-21	36.529999	36.660000	33.000000	34.029999	33.927845	168192700
2012-05-22	32.610001	33.590000	30.940001	31.000000	30.906942	101786600
2012-05-23	31.370001	32.500000	31.360001	32.000000	31.903942	73600000
2012-05-24	32.950001	33.209999	31.770000	33.029999	32.930851	50237200
2012-05-25	32.900002	32.950001	31.110001	31.910000	31.814211	37149800
2012-05-29	31.480000	31.690001	28.650000	28.840000	28.753426	78063400
2012-05-30	28.700001	29.549999	27.860001	28.190001	28.105379	57267900
2012-05-31	28.549999	29.670000	26.830000	29.600000	29.511147	111639200
2012-06-01	28.889999	29.150000	27.389999	27.719999	27.636787	41855500
2012-06-04	27.200001	27.650000	26.440001	26.900000	26.819250	35230300
2012-06-05	26.700001	27.760000	25.750000	25.870001	25.792343	42473400
2012-06-06	26.070000	27.170000	25.520000	26.809999	26.729521	61489200
2012-06-07	27.000000	27.350000	26.150000	26.309999	26.231020	26159500
2012-06-08	26.549999	27.760000	26.440001	27.100000	27.018650	38034000
2012-06-11	27.180000	28.070000	26.840000	27.010000	26.928921	28219600
2012-06-12	27.480000	27.770000	26.959999	27.400000	27.317749	15816800
2012-06-13	27.660000	28.100000	27.100000	27.270000	27.188141	17102800
2012-06-14	27.650000	28.320000	27.379999	28.290001	28.205078	16855000
2012-06-15	28.510000	30.100000	28.350000	30.010000	29.919914	43563800
2012-06-18	29.959999	32.080002	29.410000	31.410000	31.315712	42978900
2012-06-19	31.540001	32.180000	30.700001	31.910000	31.814211	30849000
2012-06-20	31.920000	31.930000	31.150000	31.600000	31.505142	15553600
2012-06-21	31.670000	32.500000	31.510000	31.840000	31.744421	21875300
2012-06-22	32.410000	33.450001	32.060001	33.049999	32.950787	74834000

```
plt.figure(figsize=(15, 8))
plt.plot(df['Adj Close'])
```

Then we plot the adjacent close values into a plot using **pyplot**,



```
df['Returns'] = df['Adj Close'].pct_change(60).shift(-60)
list_of_feture = ['High', 'Low', 'Close', 'Volume', 'Adj Close']
X = df[list_of_feture]
y = np.where(df['Returns'] > 0, 1, 0)
```

Then I created a **returns** column / feature for the data frame, this “returns” values are calculated based on the change of percentage shifting through 60 days (**lower the days, lower the accuracy on the decision tree**)

And the prediction is NOT based on a single feature the prediction is based on all the features except the ‘OPEN column / feature’

The y value will be calculated with a condition statement.

	Open	High	Low	Close	Adj Close	Volume	Returns
Date							
2012-05-18	42.049999	45.000000	38.000000	38.230000	38.115242	573576400	-0.466911
2012-05-21	36.529999	36.660000	33.000000	34.029999	33.927845	168192700	-0.377020
2012-05-22	32.610001	33.590000	30.940001	31.000000	30.906942	101786600	-0.359032
2012-05-23	31.370001	32.500000	31.360001	32.000000	31.903942	73600000	-0.404688
2012-05-24	32.950001	33.209999	31.770000	33.029999	32.930851	50237200	-0.394187
2012-05-25	32.900002	32.950001	31.110001	31.910000	31.814211	37149800	-0.399561
2012-05-29	31.480000	31.690001	28.650000	28.840000	28.753426	78063400	-0.325936
2012-05-30	28.700001	29.549999	27.860001	28.190001	28.105379	57267900	-0.310394
2012-05-31	28.549999	29.670000	26.830000	29.600000	29.511147	111639200	-0.344257
2012-06-01	28.889999	29.150000	27.389999	27.719999	27.636787	41855500	-0.309163
2012-06-04	27.200001	27.650000	26.440001	26.900000	26.819250	35230300	-0.281041
2012-06-05	26.700001	27.760000	25.750000	25.870001	25.792343	42473400	-0.261693
2012-06-06	26.070000	27.170000	25.520000	26.809999	26.729521	61489200	-0.287952
2012-06-07	27.000000	27.350000	26.150000	26.309999	26.231020	26159500	-0.313569
2012-06-08	26.549999	27.760000	26.440001	27.100000	27.018650	38034000	-0.345756
2012-06-11	27.180000	28.070000	26.840000	27.010000	26.928921	28219600	-0.312107
2012-06-12	27.480000	27.770000	26.959999	27.400000	27.317749	15816800	-0.308029
2012-06-13	27.660000	28.100000	27.100000	27.270000	27.188141	17102800	-0.303997
2012-06-14	27.650000	28.320000	27.379999	28.290001	28.205078	16855000	-0.335101
2012-06-15	28.510000	30.100000	28.350000	30.010000	29.919914	43563800	-0.352549

Here if the returns value is greater than 0, y will be 1 and if the returns value is less than 0 the y value will be 0

That is what's happening in this line of code

```
y = np.where(df['Returns'] > 0, 1, 0)
```

And now if we print the **Y** values you can see what I mean,

```
▷ y[0:70]
[305]
... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1,
          1, 1, 1, 1])
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)

print("X train shape:", X_train.shape)
print("Y train shape:", y_train.shape)
print("X test shape:", X_test.shape)
print("Y test shape:", y_test.shape)
```

then we do a train test split with separating 20% of the data untouched for testing purposes, I checked with 70% **TRAINING DATA**, the model significantly lost accuracy, so going with the industry standards are the best.

```
X train shape: (2503, 5)
Y train shape: (2503,)
X test shape: (626, 5)
Y test shape: (626,)
```

```
tree_classifier = DecisionTreeClassifier(max_depth=3,
min_samples_leaf=6)
tree_classifier.fit(X_train, y_train)
```

Then I fit the training data into the decision tree model to train the model and to deploy it

```
y_prediction = tree_classifier.predict(X_test)
```

✓ 0.0s

```
print(y_prediction)
```

✓ 0.0s

```
[1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 1 0 1 1 1
 1 1 1 0 1 1 1 1 1 1 0 1 0 1 0 1 1 1 1 1 1 1 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1
 1 1 1 0 1 0 1 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 0 1 1 1 1 1 1 1 1 0 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1
 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 0 0 1 1 1 0 1
 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 0 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 0 1 1 1
 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 1 1 1 0 1
 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 0 0 0 0 1 1 1 1 1 1 1 1 1 0 1 1 0 0 1 0 1
 1 0 1 1 1 1 1 0 1 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1 1 1 1 0 1 1 0 0 1 1 1 1 1 1 0 1
 1 0 1 1 1 1 1 1 0 1 0 1 0 0 1 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 0 1 0 1 1 1 0 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 1 1
 1 1 1 0 1 1 1 1 0 0 1 1 1 0 0 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1
 1 1 1 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 0 1 0 1 1 1 1 1 0 1 1 1 1 0
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1
 0 1 1 1 1 1 0 1 0 1 0 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 1 0 1 0 1 0 1 1 1 1 1 1 1
 0 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 1 1 0 1 0 0 0 1 1 0 1 1 1
 0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1 1 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 0]
```

You can see what I mean by binary prediction / classification

```
from sklearn.metrics import classification_report
report = classification_report(y_test, y_prediction)
print(report)
```

After I did the prediction, I took a classification report to check the accuracy of the decision tree model that we just tested. I got different accuracy scores each time I retrain the model.

And the accuracy is not sourly based on one score its based 3

Before going to the reports what is an f1- score, this is calculated like this

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$$

TP = number of true positives
FP = number of false positives
FN = number of false negatives

The f1 score refers to a **weighted harmonic mean of the precision and recall**
The best f1 score is 1.0 and worst being 0.0

Here is three times I tested

...	precision	recall	f1-score	support
0	0.88	0.21	0.35	228
1	0.69	0.98	0.81	398
accuracy			0.70	626
macro avg	0.78	0.60	0.58	626
weighted avg	0.75	0.70	0.64	626

In this report the precision of the sell prediction is high with a 88% accuracy score but the f1-score is bad, while this is the case for selling signals, in buying signals all though I got less precision than the selling signals it took the lead in the f1-score, overall model accuracy is 70%

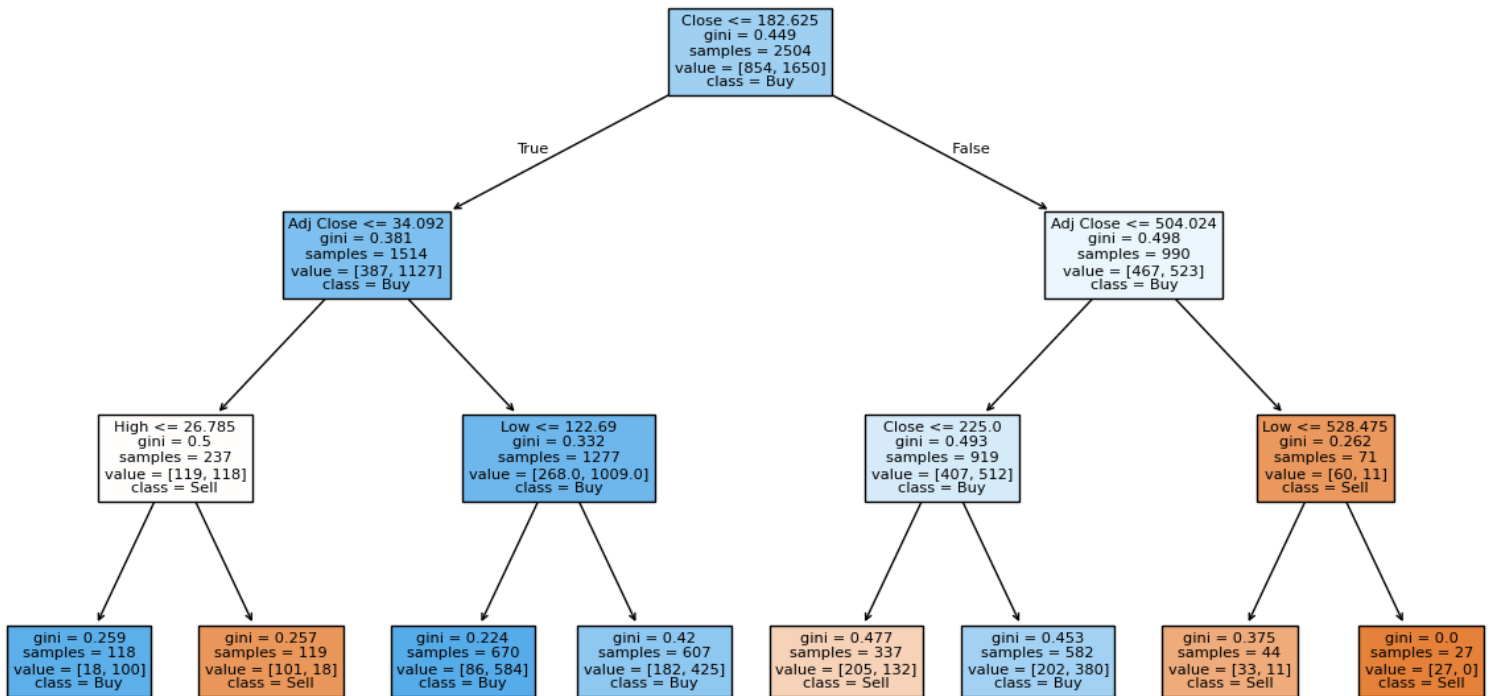
...		precision	recall	f1-score	support
	0	0.94	0.16	0.28	203
	1	0.71	1.00	0.83	423
	accuracy			0.73	626
	macro avg	0.83	0.58	0.55	626
	weighted avg	0.79	0.73	0.65	626

In this report, while the precision of the selling signals are very accurate with a 94% score, its f1 score is bad, but in the buying signals the precision is 71% and the f1 score is 83% which is good

...		precision	recall	f1-score	support
	0	0.64	0.36	0.46	198
	1	0.75	0.91	0.82	428
	accuracy			0.73	626
	macro avg	0.70	0.63	0.64	626
	weighted avg	0.72	0.73	0.71	626

In this report, the selling precision dropped to a 64% but keeping the f1 score at a mid-46% percent which I say maybe a good tread to make. And as always the buy precision is 75% with a good harmonic mean of 82% as far as I go this iteration of the model is 1 of the best I got.

Finally, the visual of the decision tree



The inspiration for this extension is taken from this journal -

https://link.springer.com/chapter/10.1007/978-3-319-69023-0_23

Przemysław Juszczuk, & Kozak, J. (2017). Classification and Preprocessing in the Stock Data. *Lecture Notes in Business Information Processing*, 269–281.

https://doi.org/10.1007/978-3-319-69023-0_23